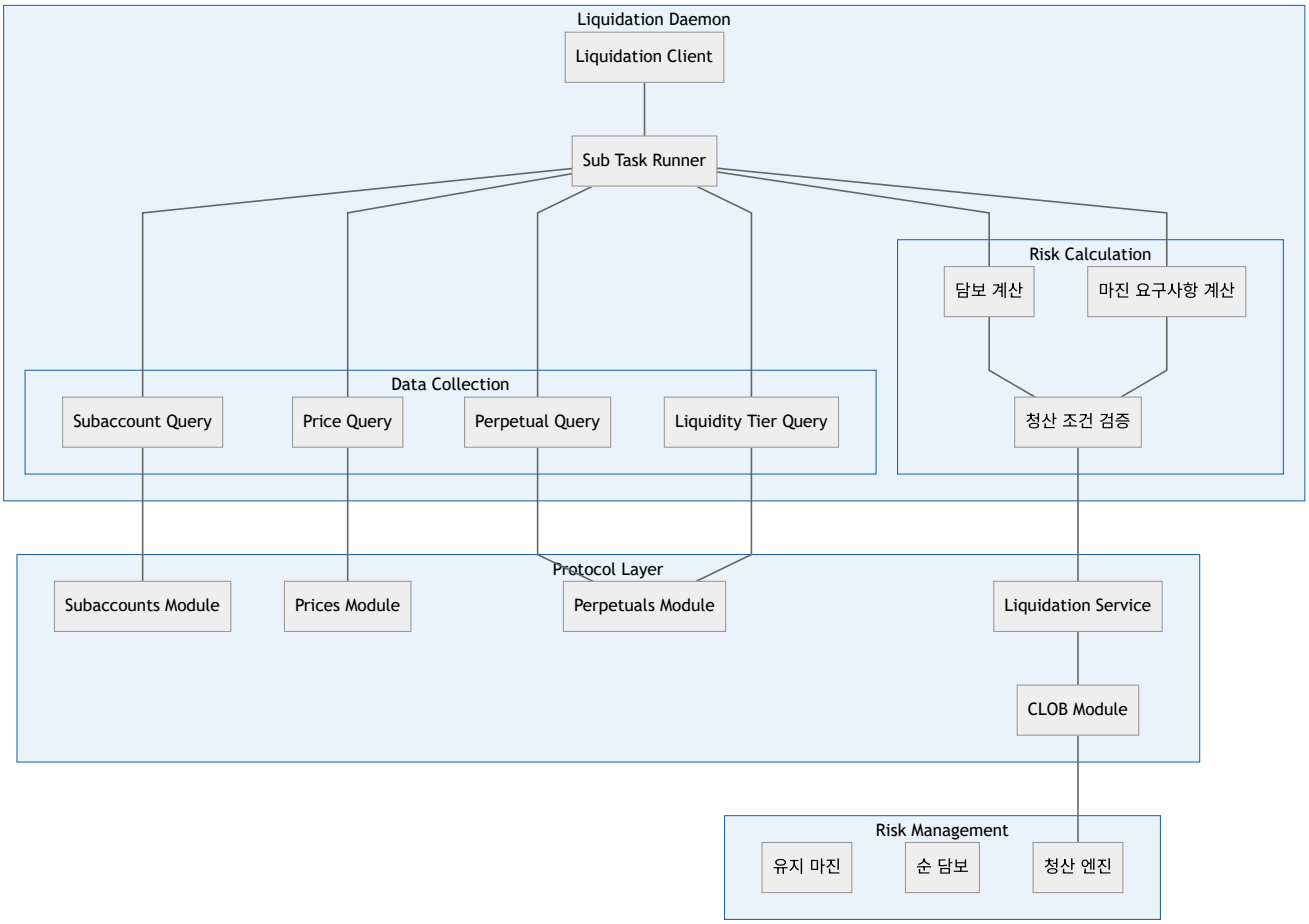
Liquidation Daemon 구현 분석

개요

Liquidation Daemon은 dYdX v4 Protocol의 핵심 리스크 관리 구성 요소로, 계정별 마진 비율을 지속적으로 모니터링하고 청산 대상 계정을 식별하여 시스템의 건전성을 유지하는 역할을 담당합니다. 이 데몬은 실시간으로 모든 서브계정의 담보 상태를 검증하고, 청산이 필요한 계정들을 Protocol에 보고하여 자동 청산 프로세스를 트리거합니다.

아키텍처 개요



계정별 마진 비율 모니터링 시스템

1. 주기적 모니터링 메커니즘

데몬 실행 주기 설정

```
func (c *Client) Start(ctx  context.Context, flags  flags.DaemonFlags, ...) erro

// 설정된 주기(기본값: 1초)로 청산 검사 실행

ticker := time.NewTicker(time.Duration(flags.Liquidation.LoopDelayMs) * time.Mil

stop := make(chan  bool)

s := &SubTaskRunnerImpl{}

StartLiquidationsDaemonTaskLoop(client, ctx, s, flags, ticker, stop)

}


func  StartLiquidationsDaemonTaskLoop(client *Client, ctx  context.Context, ...)

for {

select {

case <-ticker.C:

// 주기적으로 청산 검사 실행

if  err := s.RunLiquidationDaemonTaskLoop(ctx, client, flags.Liquidation); err !

client.logger.Error("Liquidations daemon returned error", "error", err)

client.ReportFailure(err)

} else {

client.ReportSuccess()

}

case <-stop:

return
```

```
}

}

}
```

블록 높이 기반 상태 조회

```
func (s *SubTaskRunnerImpl) RunLiquidationDaemonTaskLoop(ctx  context.Context, d

// 1. 최신 커밋된 블록 높이 조회

lastCommittedBlockHeight, err := daemonClient.GetPreviousBlockInfo(ctx)

if  err != nil {

return  err

}

// 2. 해당 블록 높이에서의 애플리케이션 상태 조회

subaccounts, marketPrices, perpetuals, liquidityTiers, err := daemonClient.Fetch

ctx,

lastCommittedBlockHeight,

liqFlags,

)

// 3. 청산 대상 계정 식별

liquidatableSubaccountIds, negativeTncSubaccountIds, err := daemonClient.GetLiqu

subaccounts,

marketPrices,

perpetuals,

liquidityTiers,
```

```
)
// 4. 청산 대상 계정 정보를 Protocol에 전송
return  daemonClient.SendLiquidatableSubaccountIds(
ctx,
lastCommittedBlockHeight,
liquidatableSubaccountIds,
negativeTncSubaccountIds,
subaccountOpenPositionInfo,
)
}
```

2. 데이터 수집 및 동기화

페이지네이션을 통한 대량 데이터 처리

```
func (c *Client) GetAllSubaccounts(ctx  context.Context, pageLimit  uint64) ([]s
subaccounts := make([]satypes.Subaccount, 0)
var  nextKey []byte
for {
// 페이지별로 서브계정 데이터 조회
subaccountsFromKey, next, err := getSubaccountsFromKey(
ctx,
c.SubaccountQueryClient,
```

```
nextKey,
pageLimit,
)
if err != nil {
return nil, err
}
subaccounts = append(subaccounts, subaccountsFromKey...)
nextKey = next
if len(nextKey) == 0 {
break // 모든 데이터 조회 완료
}
}
return subaccounts, nil
}
```

특정 블록 높이에서의 일관된 상태 조회

```
func (c *Client) FetchApplicationStateAtBlockHeight(ctx context.Context, blockH
// 모든 쿼리를 동일한 블록 높이에서 실행하여 일관성 보장
queryCtx := newContextWithQueryBlockHeight(ctx, blockHeight)
// 병렬로 필요한 모든 데이터 조회
subaccounts, err := c.GetAllSubaccounts(queryCtx, liqFlags.QueryPageLimit)
marketPrices, err := c.GetAllMarketPrices(queryCtx, liqFlags.QueryPageLimit)
```

```
perpetuals, err := c.GetAllPerpetuals(queryCtx, liqFlags.QueryPageLimit)

liquidityTiers, err := c.GetAllLiquidityTiers(queryCtx, liqFlags.QueryPageLimit)

return subaccounts, marketPricesMap, perpetualsMap, liquidityTiersMap, nil

}


func newContextWithQueryBlockHeight(ctx context.Context, blockHeight uint32)

return metadata.NewOutgoingContext(

ctx,

metadata.Pairs(grpc.GRPCBlockHeightHeader, fmt.Sprintf("%d", blockHeight)),

)

}
```

청산 대상 계정 식별 알고리즘

1. 담보 및 마진 계산 로직

순 담보 (Net Collateral) 계산

```
func (c *Client) CheckSubaccountCollateralization(

unsettledSubaccount satypes.Subaccount,

marketPrices map[uint32]pricestypes.MarketPrice,

perpetuals map[uint32]perptypes.Perpetual,

liquidityTiers map[uint32]perptypes.LiquidityTier,

) (isLiquidatable bool, hasNegativeTnc bool, err error) {
```

```
// 1. 펀딩 페이먼트를 포함한 정산된 계정 상태 계산
settledSubaccount, _, err := sakeeper.GetSettledSubaccountWithPerpetuals(
unsettledSubaccount,
perpetuals,
)
bigTotalNetCollateral := big.NewInt(0)
bigTotalMaintenanceMargin := big.NewInt(0)
// 2. 자산 포지션의 순 담보 계산 (현재는 USDC만 지원)
for _, assetPosition := range settledSubaccount.AssetPositions {
if assetPosition.AssetId != assetstypes.AssetUsdc.Id {
return false, false, errorsmod.Wrapf(
assetstypes.ErrNotImplementedMulticollateral,
"Asset %d is not supported",
assetPosition.AssetId,
)
}
// USDC의 순 담보는 포지션의 quantums 값
bigTotalNetCollateral.Add(bigTotalNetCollateral, assetPosition.GetBigQuantums())
}
// 3. 영구 선물 포지션의 순 담보 및 마진 요구사항 계산
for _, perpetualPosition := range settledSubaccount.PerpetualPositions {
perpetual := perpetuals[perpetualPosition.PerpetualId]
marketPrice := marketPrices[perpetual.Params.MarketId]
```

```
liquidityTier := liquidityTiers[perpetual.Params.LiquidityTier]

bigQuantums := perpetualPosition.GetBigQuantums()

// 포지션의 순 담보 계산 (시장가 기준)

bigNetCollateralQuoteQuantums := perpkeeper.GetNetNotionalInQuoteQuantums(

perpetual,

marketPrice,

bigQuantums,

)

bigTotalNetCollateral.Add(bigTotalNetCollateral, bigNetCollateralQuoteQuantums)

// 유지 마진 요구사항 계산

_, bigMaintenanceMarginQuoteQuantums := perpkeeper.GetMarginRequirementsInQuoteQ

perpetual,

marketPrice,

liquidityTier,

bigQuantums,

)

bigTotalMaintenanceMargin.Add(bigTotalMaintenanceMargin, bigMaintenanceMarginQuo

}

// 4. 청산 조건 검증

return  clobkeeper.CanLiquidateSubaccount(bigTotalNetCollateral, bigTotalMainten

bigTotalNetCollateral.Sign() == -1, // 음수 순 담보 여부

nil
```

```
}
```

청산 조건 판단 로직

```
// CanLiquidateSubaccount는 주어진 순 담보와 유지 마진 요구사항을 기반으로
// 서브계정이 청산 가능한지 판단합니다.
//
// 다음 두 조건이 모두 참일 때 청산 가능합니다:
// - 유지 마진 요구사항이 0보다 큼 (음수가 될 수 없음)
// - 유지 마진 요구사항이 서브계정의 순 담보보다 큼
func CanLiquidateSubaccount(bigNetCollateral *big.Int, bigMaintenanceMargin *bi
return bigMaintenanceMargin.Sign() > 0 && bigMaintenanceMargin.Cmp(bigNetCollat
}
```

2. 마진 요구사항 계산

유동성 계층 기반 마진 계산

```
func GetMarginRequirementsInQuoteQuantums(
perpetual perptypes.Perpetual,
marketPrice pricestypes.MarketPrice,
liquidityTier perptypes.LiquidityTier,
bigQuantums *big.Int,
) (bigInitialMarginQuoteQuantums *big.Int, bigMaintenanceMarginQuoteQuantums *bi
```

```
// 포지션의 명목 가치 계산
bigNotionalQuoteQuantums := GetNetNotionalInQuoteQuantums(perpetual, marketPrice
bigAbsNotionalQuoteQuantums := new(big.Int).Abs(bigNotionalQuoteQuantums)
// 초기 마진 = |명목가치| × 초기마진비율
bigInitialMarginQuoteQuantums = lib.BigIntMulPpm(
bigAbsNotionalQuoteQuantums,
liquidityTier.InitialMarginPpm,
)
// 유지 마진 = |명목가치| × 유지마진비율
bigMaintenanceMarginQuoteQuantums = lib.BigIntMulPpm(
bigAbsNotionalQuoteQuantums,
liquidityTier.MaintenanceMarginPpm,
)
return  bigInitialMarginQuoteQuantums, bigMaintenanceMarginQuoteQuantums
}
```

포지션별 위험도 평가

```
func (c *Client) GetLiquidatableSubaccountIds(
subaccounts []satypes.Subaccount,
marketPrices  map[uint32]pricestypes.MarketPrice,
perpetuals  map[uint32]perptypes.Perpetual,
liquidityTiers  map[uint32]perptypes.LiquidityTier,
```

```
) (liquidatableSubaccountIds []satypes.SubaccountId, negativeTncSubaccountIds []

liquidatableSubaccountIds = make([]satypes.SubaccountId, 0)

negativeTncSubaccountIds = make([]satypes.SubaccountId, 0)

for _, subaccount := range subaccounts {

// 오픈 포지션이 없는 계정은 건너뛰기

if len(subaccount.PerpetualPositions) == 0 {

continue

}

// 계정의 담보 상태 검증

isLiquidatable, hasNegativeTnc, err := c.CheckSubaccountCollateralization(

subaccount,

marketPrices,

perpetuals,

liquidityTiers,

)

if err != nil {

return nil, nil, err

}

if isLiquidatable {

liquidatableSubaccountIds = append(liquidatableSubaccountIds, *subaccount.Id)

}

if hasNegativeTnc {

negativeTncSubaccountIds = append(negativeTncSubaccountIds, *subaccount.Id)
```

```
}
}
return liquidatableSubaccountIds, negativeTncSubaccountIds, nil
}
```

청산 주문 생성 및 실행 프로세스

1. 청산 대상 정보 전송

gRPC를 통한 청산 요청

```
func (c *Client) SendLiquidatableSubaccountIds(
ctx context.Context,
blockHeight uint32,
liquidatableSubaccountIds []satypes.SubaccountId,
negativeTncSubaccountIds []satypes.SubaccountId,
openPositionInfoMap map[uint32]*clobtypes.SubaccountOpenPositionInfo,
) error {
// 오픈 포지션 정보를 슬라이스로 변환 (정렬된 순서로)
sortedPerpetualIds := lib.GetSortedKeys[lib.Sortable[uint32]](openPositionInfoMa
subaccountOpenPositionInfo := make([]clobtypes.SubaccountOpenPositionInfo, 0)
for _, perpetualId := range sortedPerpetualIds {
subaccountOpenPositionInfo = append(subaccountOpenPositionInfo, *openPositionInf
```

```
}

// gRPC 요청 생성

request := &api.LiquidateSubaccountsRequest{

BlockHeight: blockHeight,

LiquidatableSubaccountIds: liquidatableSubaccountIds,

NegativeTncSubaccountIds: negativeTncSubaccountIds,

SubaccountOpenPositionInfo: subaccountOpenPositionInfo,

}

// Protocol의 청산 서비스로 전송

_, err := c.LiquidationServiceClient.LiquidateSubaccounts(ctx, request)

return  err

}
```

오픈 포지션 정보 수집

```
func (c *Client) GetSubaccountOpenPositionInfo(

subaccounts []satypes.Subaccount,

) map[uint32]*clobtypes.SubaccountOpenPositionInfo {

subaccountOpenPositionInfo := make(map[uint32]*clobtypes.SubaccountOpenPositionI

for  _, subaccount := range  subaccounts {

if  len(subaccount.PerpetualPositions) == 0 {

continue

}
```

```
for _, perpetualPosition := range subaccount.PerpetualPositions {
openPositionInfo, ok := subaccountOpenPositionInfo[perpetualPosition.PerpetualId
if !ok {
openPositionInfo = &clobtypes.SubaccountOpenPositionInfo{
PerpetualId: perpetualPosition.PerpetualId,
SubaccountsWithLongPosition: make([]satypes.SubaccountId, 0),
SubaccountsWithShortPosition: make([]satypes.SubaccountId, 0),
}
subaccountOpenPositionInfo[perpetualPosition.PerpetualId] = openPositionInfo
}
// 롱/숏 포지션별로 계정 분류
if perpetualPosition.GetIsLong() {
openPositionInfo.SubaccountsWithLongPosition = append(
openPositionInfo.SubaccountsWithLongPosition,
*subaccount.Id,
)
} else {
openPositionInfo.SubaccountsWithShortPosition = append(
openPositionInfo.SubaccountsWithShortPosition,
*subaccount.Id,
)
}
}
```

```
}

return subaccountOpenPositionInfo

}
```

2. 파산 가격 계산

파산 가격 산정 알고리즘

```
func (k Keeper) GetBankruptcyPriceInQuoteQuantums(

ctx sdk.Context,

subaccountId satypes.SubaccountId,

perpetualId uint32,

deltaQuantums *big.Int,

) (deltaQuoteQuantums *big.Int, err error) {

// 파산 가격 계산 공식: -DNNV - (TNC * (abs(DMMR) / TMMR))

// 여기서:

// - DNNV: 델타 포지션 순 명목 가치

// - TNC: 총 순 담보

// - DMMR: 델타 유지 마진 요구사항

// - TMMR: 총 유지 마진 요구사항

// 1. 총 순 담보 및 총 유지 마진 요구사항 조회

tncBig, _, tmmrBig, err := k.subaccountsKeeper.GetNetCollateralAndMarginRequirem

ctx,

satypes.Update{SubaccountId: subaccountId},
```

```
)

if err != nil {

return nil, err

}

// 2. 현재 포지션 크기 조회

subaccount := k.subaccountsKeeper.GetSubaccount(ctx, subaccountId)

position, _ := subaccount.GetPerpetualPositionForId(perpetualId)

psBig := position.GetBigQuantums()

// 3. 델타 검증 (포지션 크기와 반대 방향이어야 함)

if psBig.Sign()*deltaQuantums.Sign() != -1 || psBig.CmpAbs(deltaQuantums) == -1

return nil, errorsmod.Wrapf(

types.ErrInvalidPerpetualPositionSizeDelta,

"Position size delta %v is invalid for position size %v",

deltaQuantums,

psBig,

)

}

// 4. 파산 가격 계산 로직 실행

// (구체적인 계산 로직은 복잡한 수학적 공식을 포함)

return deltaQuoteQuantums, nil

}
```

리스크 관리 정책 및 구현 방식

1. 다층 리스크 관리 체계

마진 계층 구조

```
type LiquidityTier struct {
Id uint32 // 유동성 계층 ID
Name string // 계층 이름
InitialMarginPpm uint32 // 초기 마진 비율 (ppm 단위)
MaintenanceMarginPpm uint32 // 유지 마진 비율 (ppm 단위)
BasePositionNotional uint64 // 기본 포지션 명목가치
ImpactNotional uint64 // 임팩트 명목가치
}

// 예시: BTC 유동성 계층
// InitialMarginPpm: 50000 (5%)
// MaintenanceMarginPpm: 30000 (3%)
```

위험도별 차등 마진 정책

```
func GetMarginRequirementsInQuoteQuantums(
perpetual perptypes.Perpetual,
marketPrice pricestypes.MarketPrice,
liquidityTier perptypes.LiquidityTier,
bigQuantums *big.Int,
) (bigInitialMarginQuoteQuantums *big.Int, bigMaintenanceMarginQuoteQuantums *bi
```

```
// 포지션 크기에 따른 차등 마진 적용
bigNotionalQuoteQuantums := GetNetNotionalInQuoteQuantums(perpetual, marketPrice
bigAbsNotionalQuoteQuantums := new(big.Int).Abs(bigNotionalQuoteQuantums)
// 기본 마진 계산
bigInitialMarginQuoteQuantums = lib.BigIntMulPpm(
bigAbsNotionalQuoteQuantums,
liquidityTier.InitialMarginPpm,
)
bigMaintenanceMarginQuoteQuantums = lib.BigIntMulPpm(
bigAbsNotionalQuoteQuantums,
liquidityTier.MaintenanceMarginPpm,
)
// 대형 포지션에 대한 추가 마진 요구사항 적용 가능
// (구현에 따라 포지션 크기별 스케일링 팩터 적용)
return  bigInitialMarginQuoteQuantums, bigMaintenanceMarginQuoteQuantums
}
```

2. 실시간 리스크 모니터링

성능 메트릭 수집

```
func (c *Client) CheckSubaccountCollateralization(...) (bool, bool, error) {
defer  telemetry.ModuleMeasureSince(
```

```
metrics.LiquidationDaemon,
time.Now(),
metrics.CheckCollateralizationForSubaccounts,
metrics.Latency,
)
// 담보 계산 로직...
// 청산 대상 계정 수 메트릭 기록
telemetry.ModuleSetGauge(
metrics.LiquidationDaemon,
float32(len(liquidatableSubaccountIds)),
metrics.LiquidatableSubaccountIds,
metrics.Count,
)
return  isLiquidatable, hasNegativeTnc, nil
}
```

시스템 건전성 추적

```
func (s *SubTaskRunnerImpl) RunLiquidationDaemonTaskLoop(...) error {
defer  telemetry.ModuleMeasureSince(
metrics.LiquidationDaemon,
time.Now(),
metrics.MainTaskLoop,
metrics.Latency,
```

```
)
// 청산 검사 실행...
// 오픈 포지션을 가진 계정 수 추적
telemetry.ModuleSetGauge(
metrics.LiquidationDaemon,
float32(numSubaccountsWithOpenPositions),
metrics.SubaccountsWithOpenPositions,
metrics.Count,
)
return nil
}
```

3. 에러 처리 및 복구 메커니즘

견고한 에러 처리

```
func StartLiquidationsDaemonTaskLoop(client *Client, ctx context.Context, ...)
for {
select {
case <-ticker.C:
if err := s.RunLiquidationDaemonTaskLoop(ctx, client, flags.Liquidation); err !
// 에러 로깅 및 헬스체크 실패 보고
client.logger.Error("Liquidations daemon returned error", "error", err)
client.ReportFailure(err)
```

```
} else {
// 성공적인 실행 보고
client.ReportSuccess()
}
case <-stop:
return
}
}
}
```

데이터 일관성 보장

```
func (c *Client) FetchApplicationStateAtBlockHeight(ctx  context.Context, blockH
// 모든 쿼리를 동일한 블록 높이에서 실행하여 일관성 보장
queryCtx := newContextWithQueryBlockHeight(ctx, blockHeight)
// 병렬 데이터 수집 시 에러 발생 시 전체 롤백
if  err := c.collectAllData(queryCtx, liqFlags); err != nil {
return  nil, nil, nil, nil, fmt.Errorf("failed to collect consistent state: %w",
}
return  subaccounts, marketPricesMap, perpetualsMap, liquidityTiersMap, nil
}
```

결론

Liquidation Daemon은 dYdX v4의 리스크 관리 시스템의 핵심으로서 다음과 같은 주요 특징을 가집니다:

1. **실시간 모니터링**: 1초 주기로 모든 계정의 담보 상태를 지속적으로 감시
2. **정확한 리스크 계산**: 펀딩 페이먼트, 시장 가격, 유동성 계층을 모두 고려한 정밀한 마진 계산
3. **효율적인 데이터 처리**: 페이지네이션과 병렬 처리를 통한 대량 데이터 처리 최적화
4. **견고한 시스템 설계**: 에러 처리, 메트릭 수집, 헬스체크를 통한 안정적인 운영
5. **유연한 리스크 정책**: 유동성 계층별 차등 마진 정책으로 시장 상황에 적응

이러한 설계를 통해 Liquidation Daemon은 탈중앙화 거래소의 핵심 요구사항인 시스템 건전성과 사용자 자산 보호를 효과적으로 달성합니다.